



BASIC MACHINE LEARNING

Session II

A Beginner-Friendly Continuation of Machine Learning

Decision Trees • Random Forest • Clustering • Feature Engineering • Overfitting & Optimization

Mr. Shridhar Galande

What We Will Cover Today

1

Recap of Session 1

Where we left off



2

Decision Trees & Random Forest

Splitting rules to ensembles



3

Clustering (K-Means)

Our first unsupervised algorithm



4

Feature Engineering

Better inputs, better models



5

Overfitting & Underfitting

The bias-variance tradeoff



6

Optimization & Loss Functions

How models actually learn



7

Hands-on Practicals

After every concept



By the End of This Session, You Will Be Able To

- ❖ Explain **Decision Trees and Random Forests**, and how ensembles improve on a single tree
- ❖ Differentiate **supervised and unsupervised learning**
- ❖ Apply **clustering** for segmentation problems
- ❖ Perform basic **feature engineering** to improve model learning
- ❖ Identify **overfitting and underfitting** using the bias-variance tradeoff
- ❖ Understand **optimization** and gradient descent concepts
- ❖ Explain the role of **loss functions** in machine learning



Why It Matters

These ideas let models handle messier, unlabeled, real-world data — and know when to trust their own predictions.

Recap of Session 1

- ❖ Machine Learning learns rules from data, and splits into **supervised, unsupervised, and reinforcement learning**
- ❖ We always split data into **training and testing** parts, and use metrics (accuracy, precision, recall, F1, RMSE, R^2) to judge a model
- ❖ Linear Regression **predicts numbers using a best-fit line**; Logistic Regression **predicts categories using a probability or a maximum-margin boundary**
- ❖ Today, we build on this foundation with models that handle more complex, real-world, and unlabeled data

Supervised vs. Unsupervised Learning

Session 1 focused on supervised learning. Today’s Clustering practical is our first unsupervised algorithm — here is how they compare.

	Supervised Learning	Unsupervised Learning
Labels required?	Yes — each example has a known answer	No — the data has no labeled answer
Goal	Predict a known target as accurately as possible	Discover hidden structure or groupings
Example algorithms	Linear/Logistic Regression, SVM, Decision Trees	K-Means, Hierarchical Clustering, DBSCAN
Example use case	Predicting house prices; classifying tumors	Customer segmentation; anomaly detection

PART 1 • TREE-BASED MODELS

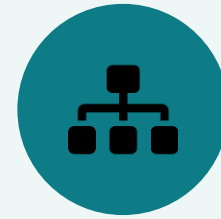
Decision Trees & Random Forest



Decision Trees

How splitting rules turn data into predictions

- ❖ Entropy & Information Gain
- ❖ Gini Impurity
- ❖ Worked examples: Play Tennis
- ❖ Overfitting, pruning & the road to Random Forest

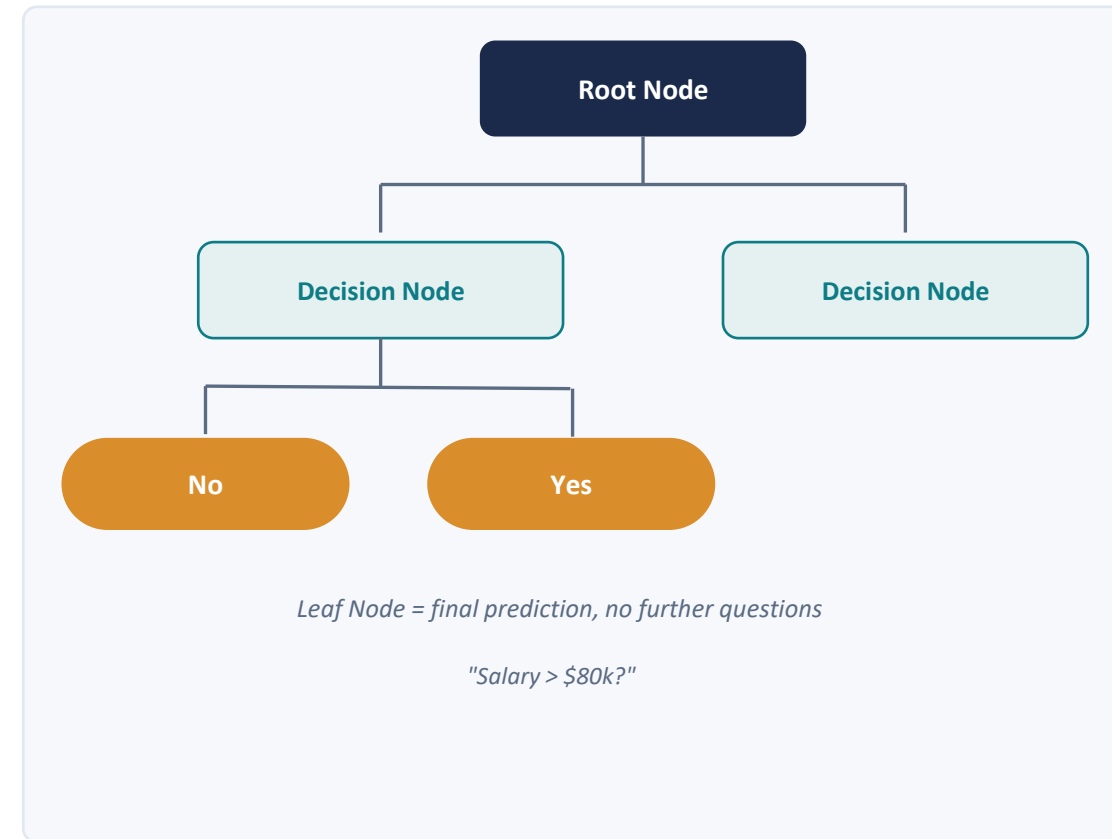


A flow-chart-like model that asks a series of simple questions to reach a prediction.

What Is a Decision Tree?

A supervised learning algorithm that predicts an outcome through a series of simple, sequential decisions.

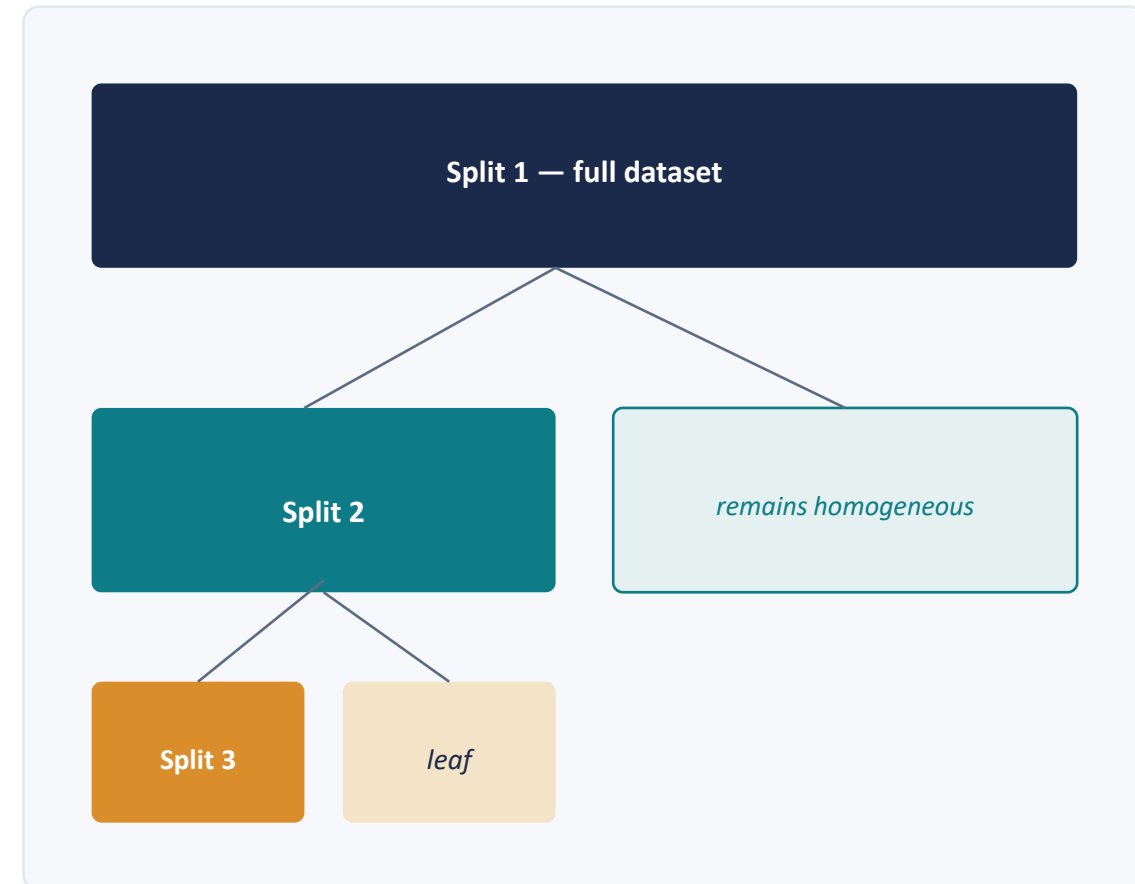
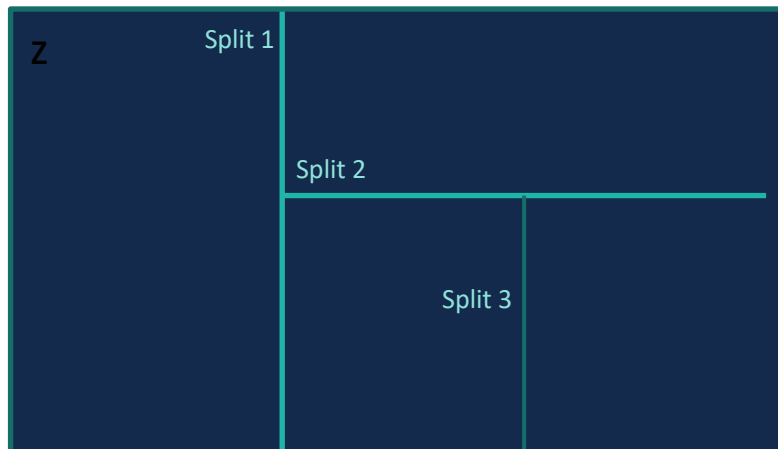
- ✓ Interpretable — **predictions can be traced as a clear chain of if-then decisions**
- ✓ Non-linear — **captures complex, non-linear relationships between features and target**
- ✓ Versatile — **handles both classification (class labels) and regression (numeric values)**
- ✓ Minimal preprocessing — **no feature scaling required; handles numeric and categorical data**



Divide and Conquer

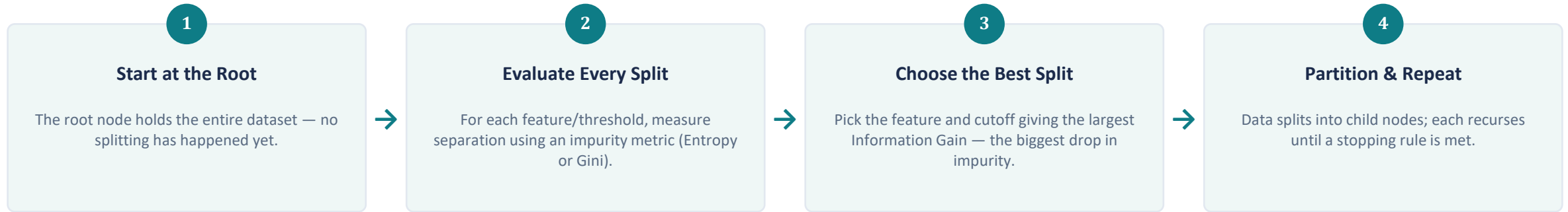
The recursive strategy that grows every decision tree.

- ✓ Also called **recursive partitioning** — split the data, then split each subset again
- ✓ Repeats until subsets are sufficiently homogeneous, or a **stopping rule** is met
- ✓ The same logic applies whether the tree is predicting a **class** or a **number**



How a Decision Tree Works

Growing a tree by recursively splitting data on the feature that best separates the classes.



Stopping Rule Stops when a node is pure, a max depth is reached, or too few samples remain to split.

Entropy: Measuring Uncertainty

Entropy measures the uncertainty or randomness in a dataset — how mixed the classes are in a node.

$$H(S) = - \sum p_i \log_2(p_i)$$

S = samples at a node
p_i = probability of class i in S
log base 2 → entropy measured in bits

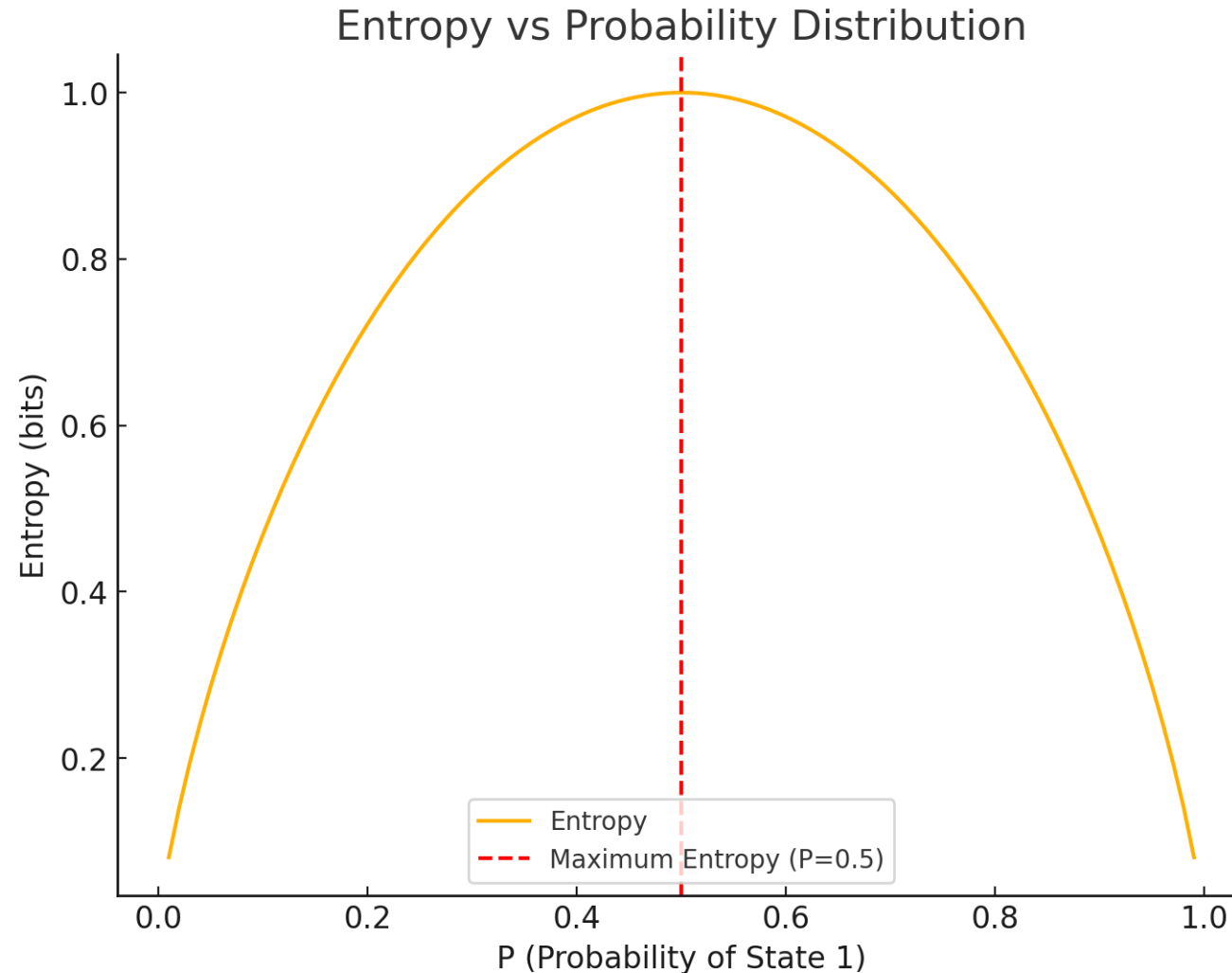
- ✓ Minimum Entropy **H(S) = 0** — **when all samples belong to one class (complete certainty)**
- ✓ Maximum Entropy **H(S) = log₂(c)** — **when classes are equally distributed**
- ✓ Entropy is **always non-negative**: **H(S) ≥ 0**

Entropy Values (Binary)

p (class 1)	H(S) bits
0.0	0.000
0.2	0.722
0.5	1.000
0.8	0.722
1.0	0.000

Entropy vs. Probability

For a binary classification problem, entropy peaks when classes are perfectly balanced.



Key Takeaways

- ✓ Entropy is **0** when the node is pure (all samples belong to one class)
- ✓ Entropy is **maximum (1 bit)** when both classes are equally likely ($p = 0.5$)
- ✓ The curve is **symmetric** around $p = 0.5$
- ✓ Higher entropy means **higher uncertainty**

Information Gain

Information Gain (IG) measures the reduction in entropy (uncertainty) after we split the dataset on an attribute.

$$IG(S, A) = H(S) - \sum \frac{|S_v|}{|S|} \cdot H(S_v)$$

S = parent node • A = attribute used for split • S_v = subset of samples with value v of A



Higher IG

More reduction in uncertainty. Better, more informative split.



Lower IG

Less reduction in uncertainty. Not a very useful split.



IG = 0

No reduction — the attribute provides no information.



Key Takeaway Decision Trees choose the split with the HIGHEST Information Gain at each step.

Gini Impurity

Gini Impurity measures the probability of misclassifying a randomly chosen sample if it were labeled according to the class distribution in that node.

$$Gini(S) = 1 - \sum p_i^2$$

- S = Dataset (or node)
- C = number of classes
- p_i = Probability (proportion) of class i

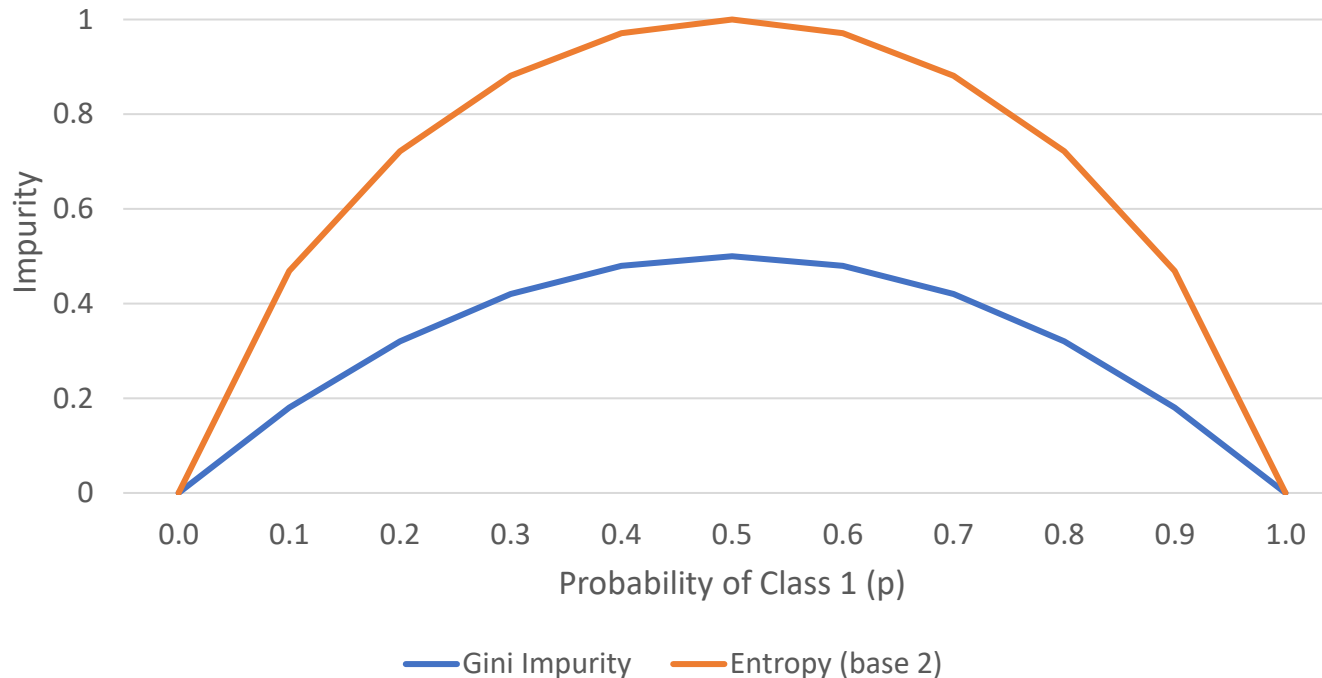
- ✓ Range: **$0 \leq Gini(S) < 1$**
- ✓ Minimum: **Gini = 0 when one class has probability 1 (pure node)**
- ✓ Maximum (binary): **Gini = 0.5 when $p_1 = p_2 = 0.5$**
- ✓ Maximum (Multi-class): $1 - \frac{1}{C}$ where C is the number of classes. This occurs when all classes are equally likely.
- ✓ Interpretation: **Lower Gini is better**—a lower Gini value means the node is purer.

Used in CART (Classification and Regression Trees)

- ✓ Computationally efficient — **only sums and squares, no logarithms**
- ✓ Works well in practice and is **widely used in CART**
- ✓ Produces **balanced and compact** trees
- ✓ Effective for both **binary and multi-class** problems

Gini Impurity vs. Entropy

Both are 0 at the extremes (pure nodes) and highest when a node is most mixed — but they behave slightly differently.



Observations

- ✓ Both are 0 at $p = 0$ and $p = 1$
- ✓ Entropy has a higher max (~0.69 in natural log terms / 1 bit) than Gini (0.5)
- ✓ Gini is **computationally simpler** (no log operations)
- ✓ Both prefer purer splits

WORKED EXAMPLE

The Play Tennis Dataset

14 days of weather observations — will tennis be played?

TARGET

Play Tennis? (Yes / No)

9 days Yes, 5 days No

FEATURES

Outlook → *Sunny / Overcast / Rain*
Temperature → *Hot / Mild / Cool*
Humidity → *High / Normal*
Windy → *True / False*

14 total days: 9 days tennis was played, 5 it was not. We'll use this small dataset to compute entropy and information gain by hand.

WORKED EXAMPLE

Play Tennis — Raw Data

Outlook	Temperature	Humidity	Windy	Play Tennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Entropy at the Source

Before any split, how uncertain is the outcome across all 14 days?

14

Total days

9

Played (Yes) $P = 9/14 = 0.643$

5

Not played (No) $P = 5/14 = 0.357$

Entropy at Source

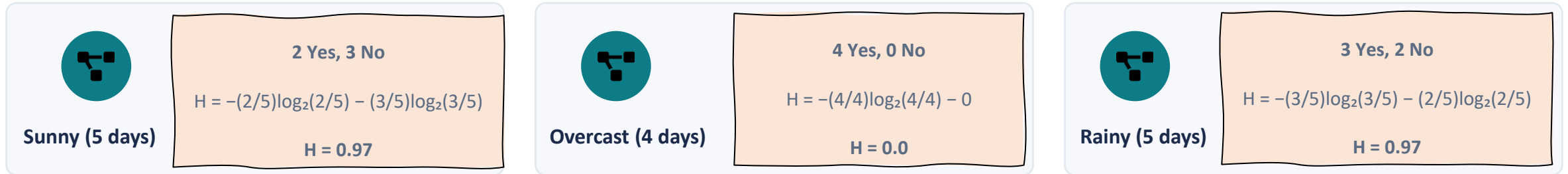
$$H(S) = -(0.643)\log_2(0.643) - (0.357)\log_2(0.357)$$

$$H(S) = 0.940 \text{ bits}$$

This is the baseline uncertainty before we split on any feature.

Splitting on “Outlook”

Entropy is computed separately within each branch, then combined as a weighted average.



Weighted Entropy of Branches

$$(5/14)(0.97) + (4/14)(0.0) + (5/14)(0.97) = 0.69$$

$$\text{Information Gain}(\text{Outlook}) = 0.940 - 0.69 = 0.246$$

The largest drop in uncertainty of any feature — Outlook is the strongest first split.

Splitting on “Humidity”

Entropy is computed separately within each branch, then combined as a weighted average.



High (7 days)

3 Yes, 4 No

$$H = -(3/7)\log_2(3/7) - (4/7)\log_2(4/7)$$

$$H = 0.99$$



Normal (7 days)

6 Yes, 1 No

$$H = -(6/7)\log_2(6/7) - (1/7)\log_2(1/7)$$

$$H = 0.59$$

Weighted Entropy of Branches

$$(7/14)(0.99) + (7/14)(0.59) = 0.79$$

$$\text{Information Gain(Humidity)} = 0.940 - 0.79 = \mathbf{0.151}$$

The second-strongest split — Humidity cleanly separates most Normal-humidity days as Yes.

Splitting on “Windy”

Entropy is computed separately within each branch, then combined as a weighted average.



Weak (8 days)

6 Yes, 2 No

$$H = -(6/8)\log_2(6/8) - (2/8)\log_2(2/8)$$

$$H = 0.81$$



Strong (6 days)

3 Yes, 3 No

$$H = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6)$$

$$H = 1.00$$

Weighted Entropy of Branches

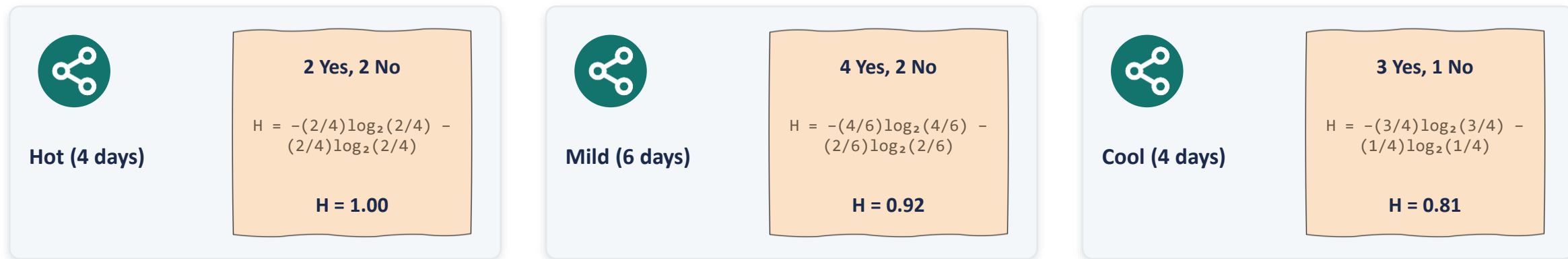
$$(8/14)(0.81) + (6/14)(1.00) = 0.89$$

$$\text{Information Gain(Windy)} = 0.940 - 0.89 = 0.048$$

The weakest split of the four — Windy barely reduces uncertainty on its own.

Splitting on “Temperature”

Entropy is computed separately within each branch, then combined as a weighted average.



Weighted Entropy of Branches

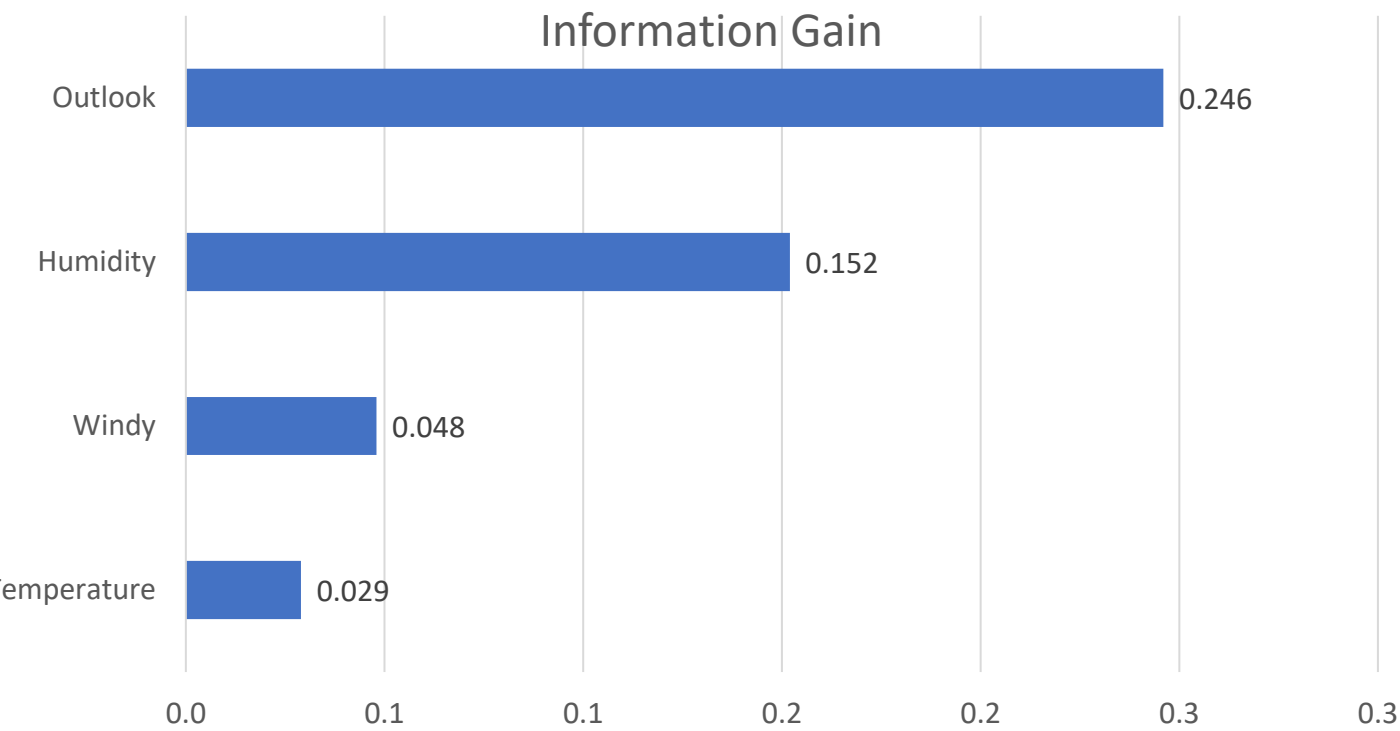
$$(4/14)(1.00) + (6/14)(0.92) + (4/14)(0.81) = 0.91$$

$$\text{Information Gain(Temperature)} = 0.940 - 0.91 = 0.029$$

The weakest of all four features — Temperature contributes the least to reducing uncertainty.

Comparing Information Gain

Repeating the calculation for every feature shows which one splits the data best.



Outlook Wins

With the highest Information Gain (0.246), Outlook is chosen as the root node — it separates the classes better than any other feature.

Gini Impurity at the Source

Before any split, how impure is the outcome across all 14 days? Using Gini Impurity.

14

Total days

9

Played (Yes) $P = 9/14 = 0.643$

5

Not played (No) $P = 5/14 = 0.357$

Gini Impurity at Source

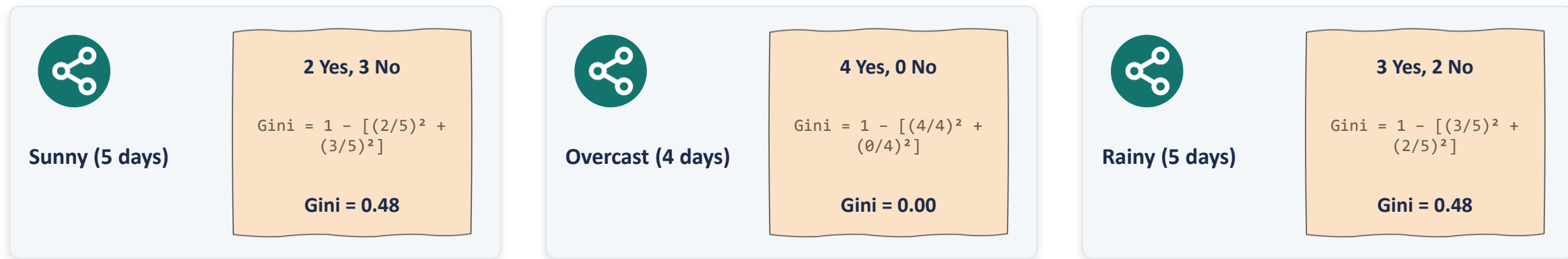
$$\text{Gini}(S) = 1 - (0.643)^2 - (0.357)^2$$

$$\text{Gini}(S) = 0.459$$

This is the baseline impurity before we split on any feature.

Splitting on “Outlook” (Gini)

Gini impurity is computed separately within each branch, then combined as a weighted average.



Weighted Gini Impurity of Branches

$$(5/14)(0.48) + (4/14)(0.00) + (5/14)(0.48) = 0.34$$

$$Gini\ Gain(Outlook) = 0.459 - 0.34 = 0.116$$

The largest drop in impurity of any feature — Outlook is the strongest first split under Gini too.

Splitting on “Humidity” (Gini)

Gini impurity is computed separately within each branch, then combined as a weighted average.



High (7 days)

3 Yes, 4 No

$$\text{Gini} = 1 - [(3/7)^2 + (4/7)^2]$$

Gini = 0.49



Normal (7 days)

6 Yes, 1 No

$$\text{Gini} = 1 - [(6/7)^2 + (1/7)^2]$$

Gini = 0.24

Weighted Gini Impurity of Branches

$$(7/14)(0.49) + (7/14)(0.24) = 0.37$$

$$\text{Gini Gain(Humidity)} = 0.459 - 0.37 = 0.092$$

The second-strongest split under Gini — Normal humidity is heavily skewed toward Yes.

Splitting on “Windy” (Gini)

Gini impurity is computed separately within each branch, then combined as a weighted average.



Weak (8 days)

6 Yes, 2 No

$$\text{Gini} = 1 - [(6/8)^2 + (2/8)^2]$$

Gini = 0.38



Strong (6 days)

3 Yes, 3 No

$$\text{Gini} = 1 - [(3/6)^2 + (3/6)^2]$$

Gini = 0.50

Weighted Gini Impurity of Branches

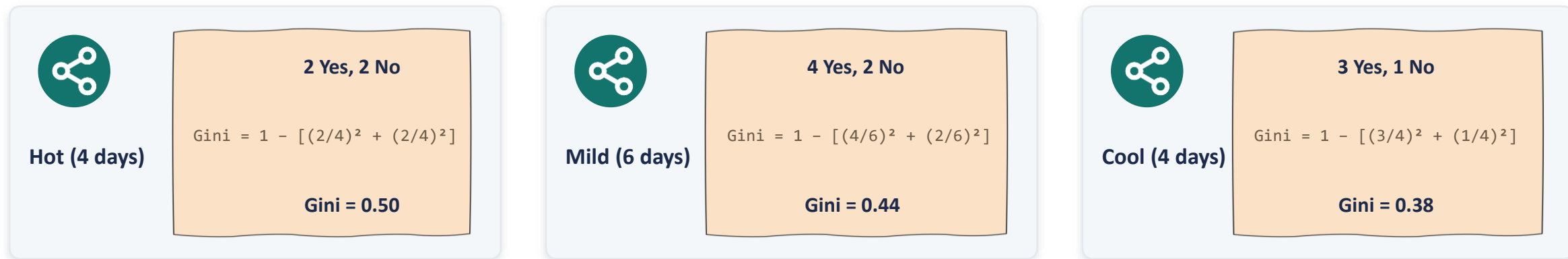
$$(8/14)(0.38) + (6/14)(0.50) = 0.43$$

$$\text{Gini Gain(Windy)} = 0.459 - 0.43 = 0.031$$

A weak split under Gini as well — Windy barely reduces impurity on its own.

Splitting on “Temperature” (Gini)

Gini impurity is computed separately within each branch, then combined as a weighted average.



Weighted Gini Impurity of Branches

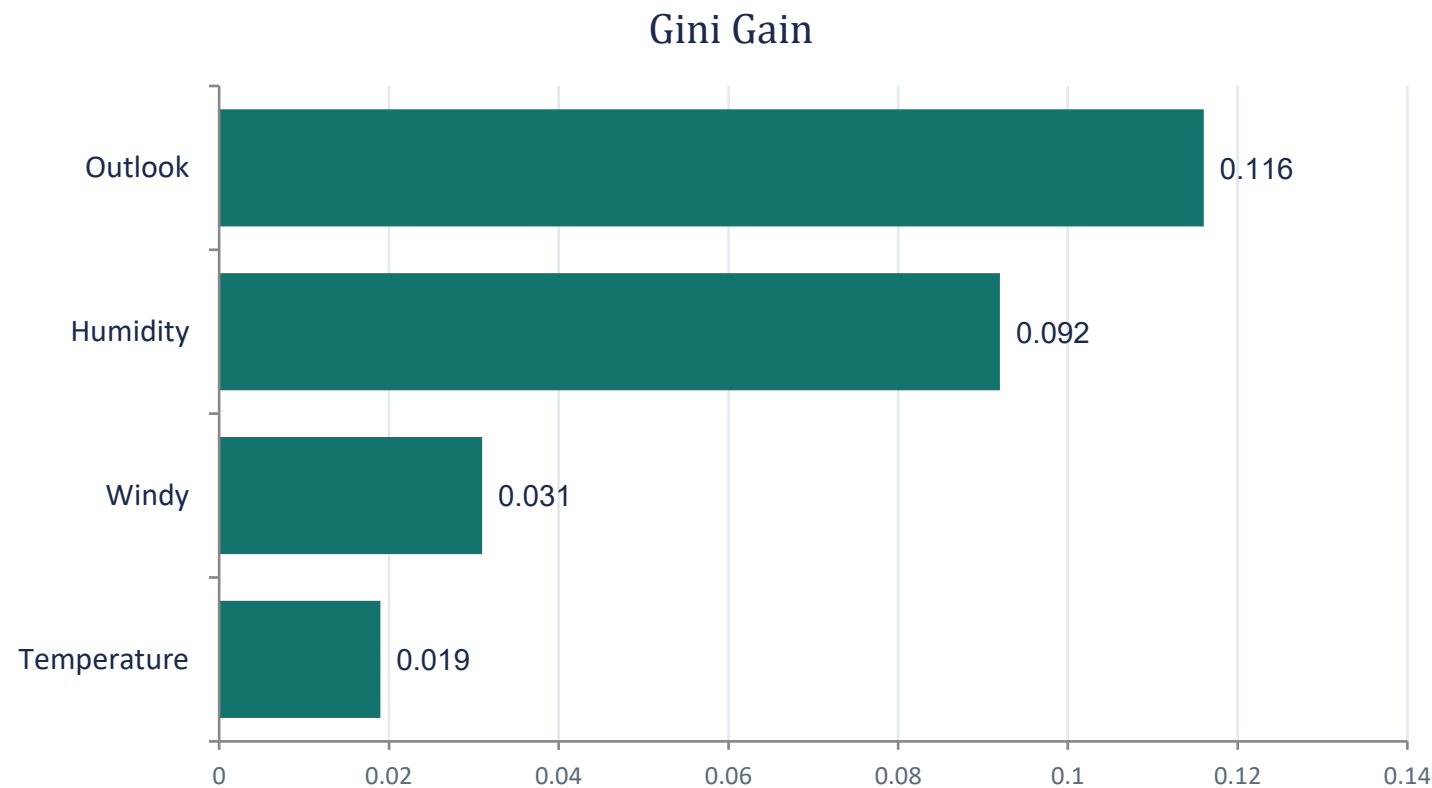
$$(4/14)(0.50) + (6/14)(0.44) + (4/14)(0.38) = 0.44$$

$$Gini\ Gain(Temperature) = 0.459 - 0.44 = 0.019$$

The weakest of all four features under Gini as well — same ranking as entropy.

Comparing Gini Gain

Repeating the calculation for every feature shows which one splits the data best. Using Gini Impurity.

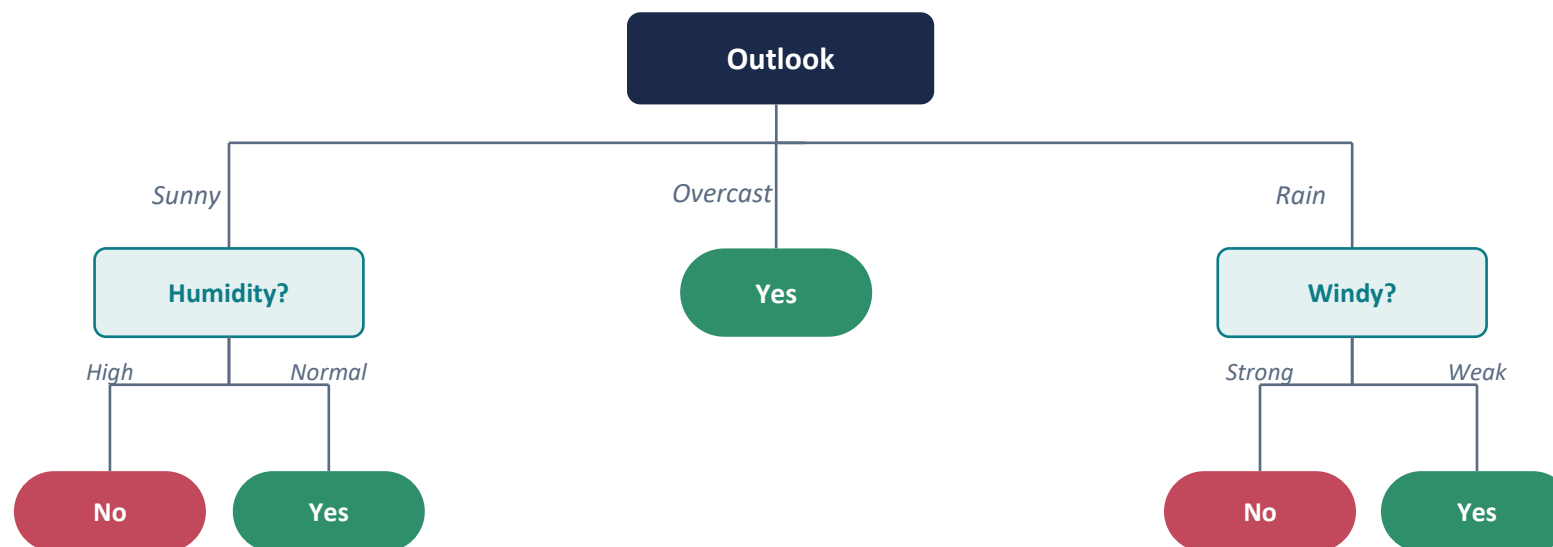


Outlook Wins

With the highest Gini Gain (0.116), Outlook is chosen as the root node — it separates the classes better than any other feature under Gini impurity as well.

The Resulting Decision Tree

Recurring on each branch (using the same Information Gain logic) produces the final tree.



Key Takeaway For a new day "Outlook = Sunny, Humidity = High" → follow the path: Outlook → Sunny → Humidity → High → Predict No.

Strengths & Limitations

Advantages

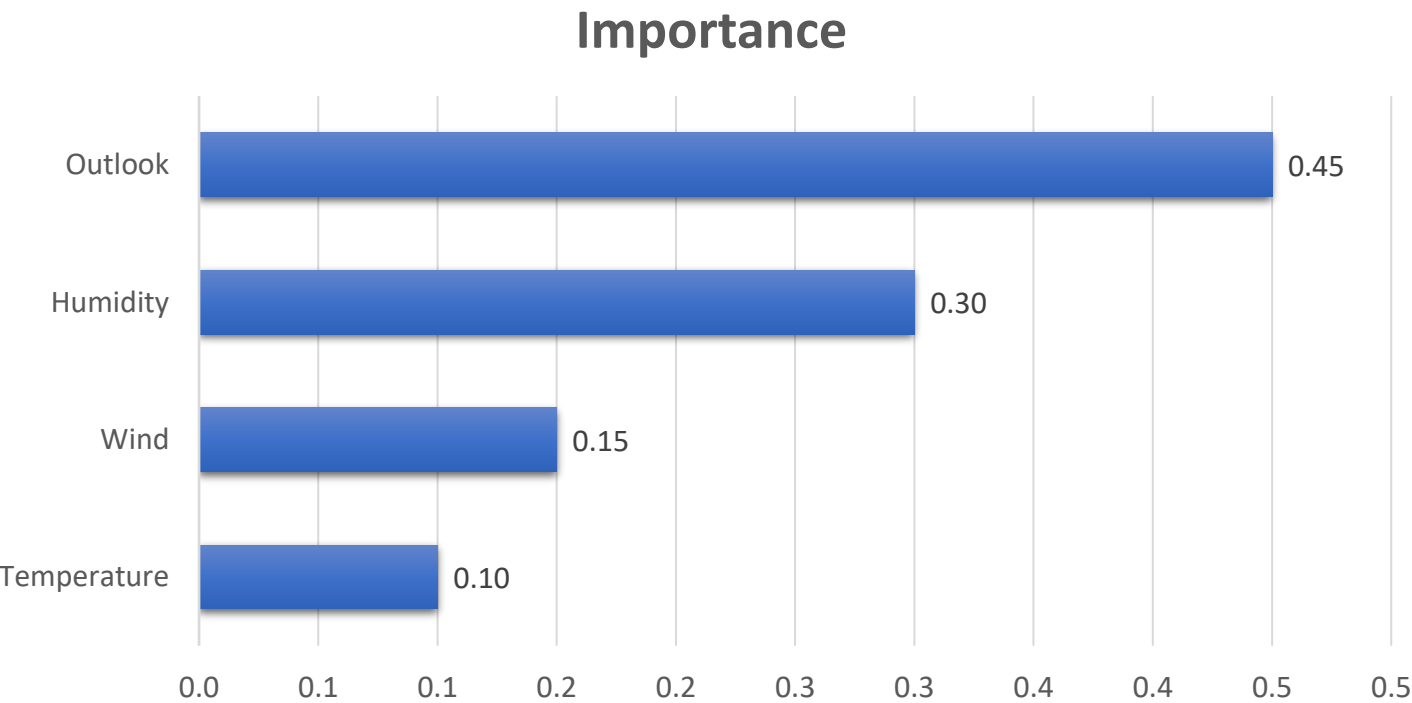
- ✓ Easy to understand and interpret
- ✓ Requires little data preprocessing
- ✓ Works with both numerical and categorical data
- ✓ Captures non-linear patterns
- ✓ Provides feature importance

Limitations

- ✓ Prone to overfitting (deep trees)
- ✓ High variance — small data changes can cause a big tree change
- ✓ Less accurate than ensemble methods (Random Forest, XGBoost)
- ✓ Biased towards features with more levels

Feature Importance

Decision Trees can tell us which features matter most in making the decision.



Insight

Outlook is the most influential feature for predicting whether tennis will be played — it carries the most weight in the tree’s decisions.

Where These Models Are Used



Credit Risk Scoring

Decide whether to approve a loan based on applicant history.



Medical Diagnosis Support

Flag likely conditions from patient symptoms and test results.



Customer Churn Prediction

Identify customers likely to cancel a subscription.



Fraud Detection

Spot unusual transactions that resemble known fraud patterns.

Why Go for Ensemble-Based Algorithms?

Decision Trees are simple and interpretable, but prone to overfitting and high variance.

- ✓ High Variance — **a tree can fit training data very well, including noise, but fail to generalize**
- ✓ Instability — **small changes in training data can create a completely different tree structure**
- ✓ Greedy Nature — **locally optimal splits which may not lead to the globally best model**
- ✓ Limited Predictive Power — **a single tree may not capture complex patterns as well as ensembles**



Key Takeaway

Ensembles reduce errors by combining many models, so the weaknesses of one tree are compensated by the strengths of others.

How Ensembles Help

Single Decision Tree: high variance. Ensemble of many trees: low variance.

- ❖ Bagging (e.g., Random Forest) → reduces variance by averaging many trees trained on different subsets of data
- ❖ Boosting (e.g., XGBoost, LightGBM, AdaBoost) → reduces bias (and variance) by focusing on hard-to-predict examples sequentially
- ❖ Stacking → combines different models/learners to get a better prediction



Higher Accuracy

Combines models to improve predictive performance.



Better Generalization

Reduces overfitting; performs well on unseen data.



Robustness

Less sensitive to noise, outliers, and small data changes.

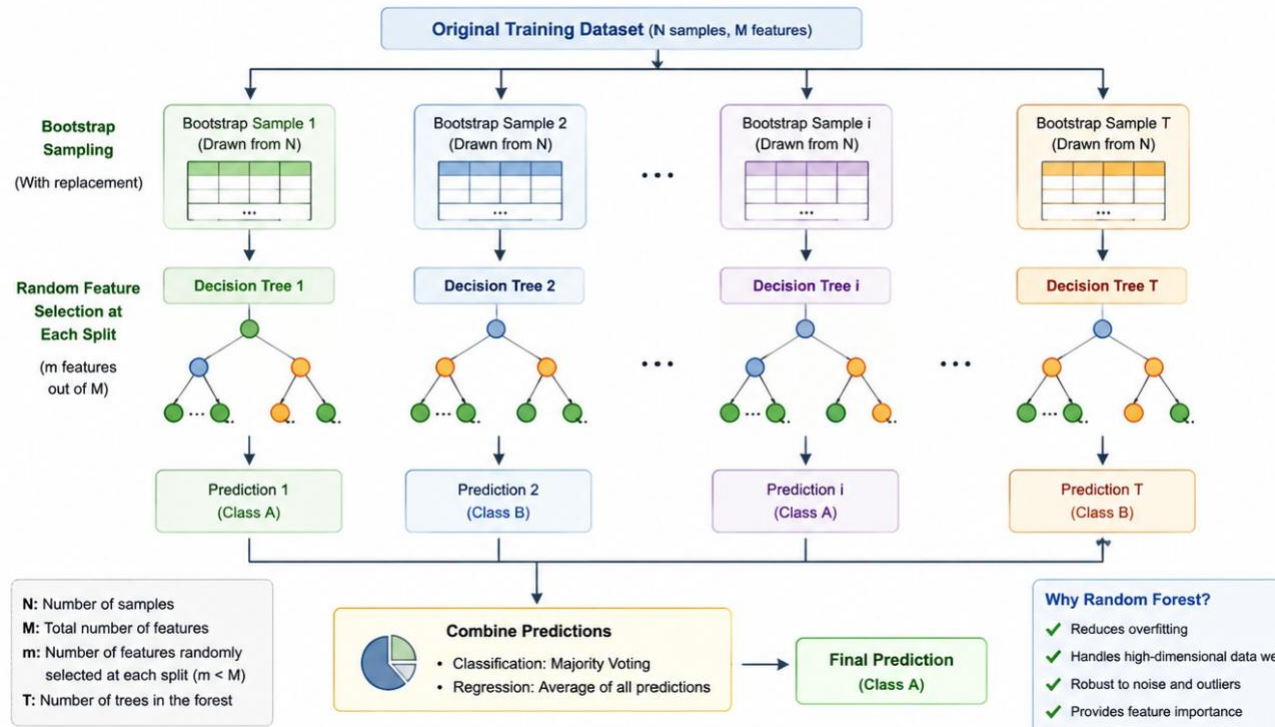


Leverages Diversity

Different trees learn different aspects; together they are stronger.

What Is Random Forest?

An ensemble learning method that builds multiple Decision Trees and outputs the majority vote (classification) or mean (regression).



Key Takeaway Combine many diverse trees to reduce overfitting and improve generalization: Decision Tree (high variance) → Random Forest (low variance, high accuracy).

Bagging + Random Feature Selection

The two mechanisms that make each tree in the forest different from the others.

Bagging (Bootstrap Aggregating)

Trains each tree on a different bootstrap sample of the data.

Random Feature Selection

At each split, considers only a random subset of features (not all).

Key Hyperparameters

Parameter	Description	Typical Values
n_estimators	Number of trees in the forest	100, 200, 500, 1000
max_depth	Maximum depth of each tree	None, 10, 20, 30
max_features	Features considered at each split	sqrt, log2, None
min_samples_split	Minimum samples required to split a node	2, 5, 10
min_samples_leaf	Minimum samples required at a leaf node	1, 2, 4

Advantages, Limitations & OOB Error

- ✓ Advantages: **high accuracy and robust to overfitting; handles missing values and outliers effectively; works well on large, high-dimensional data**
- ✓ Limitations: **less interpretable than a single tree; requires more computational power and memory; prediction time increases with more trees**

Out-of-Bag (OOB) Error

Each bootstrap sample leaves out ~37% of the data (the "out-of-bag" samples). These unseen points give a free, unbiased estimate of model performance — without needing a separate validation set.

~63%

In bootstrap sample

~37%

Out-of-bag (test)



Key Takeaway Lower OOB error → better model.

Decision Tree vs. Ensemble Methods

Aspect	Decision Tree (Single)	Ensemble Methods
Bias	Low (deep tree) / High (shallow tree)	Lower overall
Variance	High	Much lower
Overfitting Risk	High	Low
Stability	Low (sensitive to data change)	High (more stable)
Accuracy	Moderate	High
Training Time	Low	Higher
Best Use Cases	Quick insights, interpretable rules	High accuracy, production systems

HANDS-ON · PRACTICAL 1

Decision Tree & Random Forest Demo

Open the notebook: train both models, compare performance, and plot feature importance.



HANDS-ON PRACTICAL — open the notebook and follow along

PART 1 · UNSUPERVISED LEARNING

Clustering



Unsupervised Learning & Clustering

- **Unsupervised Learning** is a type of machine learning where the model learns patterns from unlabeled data.
- **Clustering** is an unsupervised technique that groups similar data points together so that points in the same cluster are more similar to each other than to those in other clusters.

Key Characteristics



No target variable

No labels are used



Discovers structure

Finds hidden patterns



Useful for segmentation

Exploration & grouping



Results may vary

With algorithm/parameters

What Is Clustering?

Clustering partitions data points into groups such that intra-cluster similarity is high and inter-cluster similarity is low.



- ✓ Intra-cluster similarity **is high** — **points within a cluster are similar**
- ✓ Inter-cluster similarity **is low** — **points in different clusters are dissimilar**
- ✓ No target variable is used to guide the grouping

CLUSTERING

How Clustering Works

A general conceptual flow, plus the distance metrics used to measure similarity.



Common Distance / Similarity Metrics

Euclidean Distance

$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

Straight-line distance

Manhattan Distance

$$d(x, y) = \sum |x_i - y_i|$$

Sum of absolute differences

Cosine Similarity

$$\text{sim}(x, y) = x \cdot y / (\|x\| \|y\|)$$

Measures orientation

Where Clustering Is Used



Customer Segmentation

Group customers by behavior for targeted marketing.



Market Basket Grouping

Find products that are frequently bought together.



Anomaly Detection

Spot points that do not belong to any well-formed cluster.



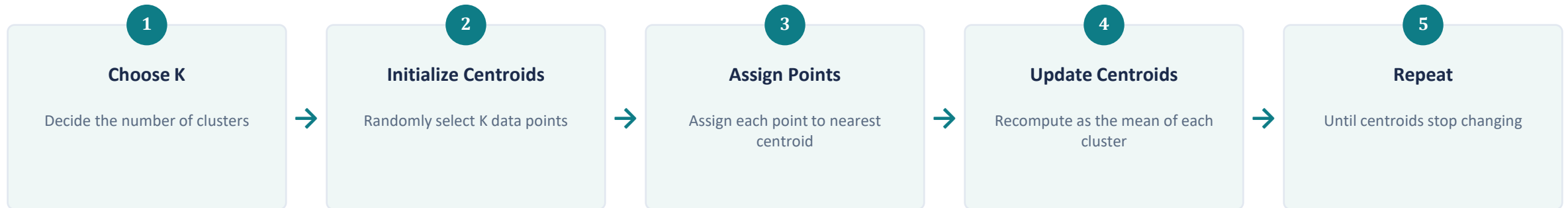
Document / Topic Grouping

Organize articles or support tickets by similar theme.

What Is K-Means?

Partition n data points into K clusters so that the within-cluster sum of squares (WCSS) is minimized.

- ✓ Requires you to specify **K** (number of clusters) in advance
- ✓ Groups points so that points in the **same cluster are close**, and points in different clusters are far apart
- ✓ Works well for **spherical, equally sized clusters**



Objective Function & Distance Metric

K-Means minimizes the total squared distance between points and their assigned cluster centroid.

$$WCSS = \sum_i \sum_{x \in c_i} \|x - \mu_i\|^2$$

C_i = set of points in cluster i • μ_i = centroid (mean) of cluster i

Distance Metric (default)

$$d(x, y) = \sqrt{\sum (x_j - y_j)^2}$$

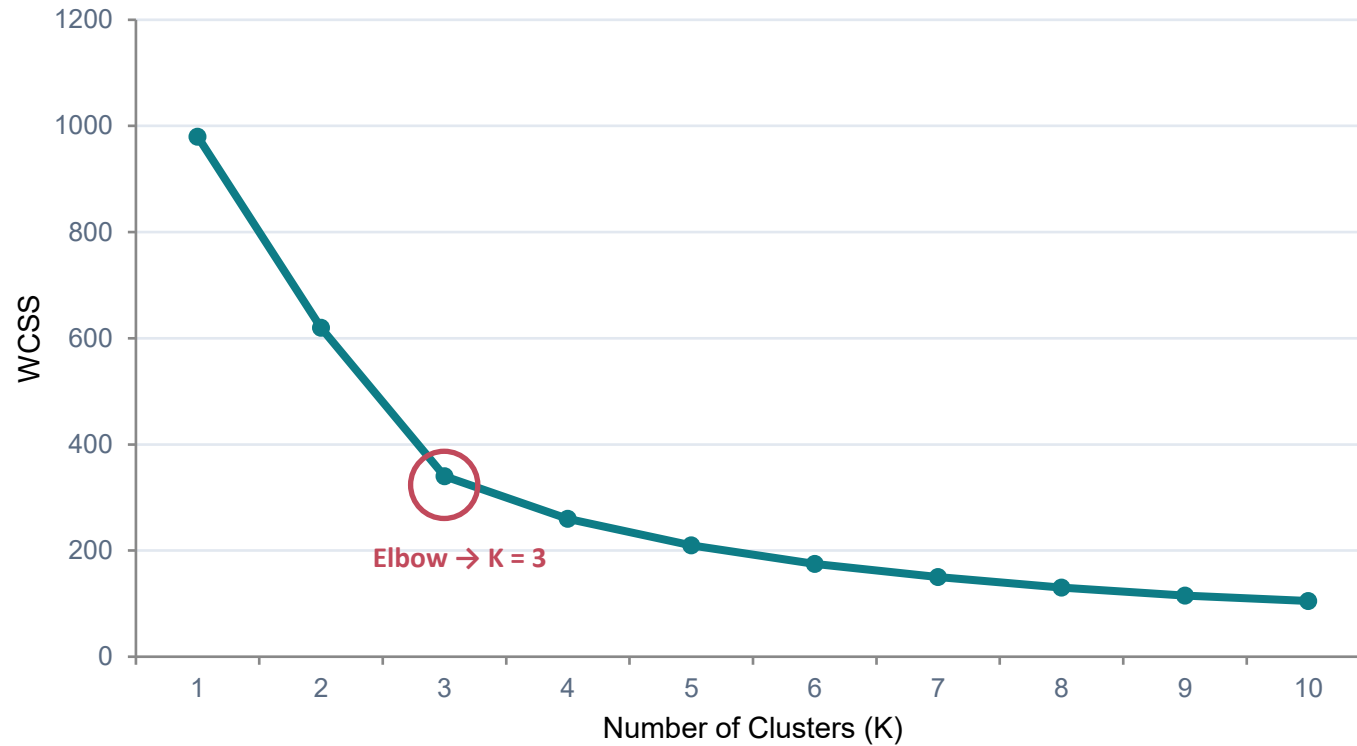
K-Means uses Euclidean distance by default; scale your data first — K-Means is sensitive to outliers and feature scale.

Algorithm (Pseudocode)

- ✓ Choose K (number of clusters)
- ✓ Initialize K centroids randomly
- ✓ Repeat until convergence: assign each point to nearest centroid, then update each centroid as the mean of its points
- ✓ Return the clusters and centroids

Choosing K — The Elbow Method

Run K-Means for different K values and plot WCSS vs. K. Choose K at the "elbow" where the decrease slows down.



Tips for Choosing K

- ✓ Visualize your data (if possible)
- ✓ Try multiple values of K
- ✓ Use domain knowledge
- ✓ Validate using metrics (Silhouette Score)

Advantages & Limitations

Advantages

- ✓ Simple and easy to implement
- ✓ Scalable to large datasets
- ✓ Fast convergence
- ✓ Works well when clusters are compact and spherical

Limitations

- ✓ Need to pre-define K
- ✓ Sensitive to initial centroids
- ✓ Not effective for non-spherical clusters or varying sizes
- ✓ Sensitive to outliers and noise

Beyond K-Means: Other Approaches



Hierarchical Clustering

Builds a tree of nested clusters (a dendrogram) by repeatedly merging or splitting groups. No need to choose K upfront — you can "cut" the tree at any level.

Best when you want to explore structure at multiple levels of granularity.



DBSCAN (Density-Based)

Groups points that are closely packed together, and automatically labels sparse points as noise/outliers. Does not require K, and handles irregularly shaped clusters.

Best when clusters are irregular in shape or outlier detection matters.

The Silhouette Score

Without labels, how do we know if a clustering is any good? For each point, the Silhouette Score compares how close it is to its own cluster versus the nearest other cluster.

$$s = (b - a) / \max(a, b)$$

a = average distance to points in the same cluster • b = average distance to points in the nearest other cluster

+0.7 to +1.0

Strong, well-separated clusters

0 to +0.5

Weak or overlapping clusters

-1.0 to 0

Points likely in the wrong cluster



Key Takeaway Averaging the score across all points gives one overall Silhouette Score for the whole clustering, on the same -1 to 1 scale.

Adjusted Rand Index (ARI)

When ground truth exists (for teaching or validation), ARI measures how well the clusters found line up with the true labels.

ARI Range: -1.0 to 1.0

- ✓ 1.0 = **perfect match with the true labels**
- ✓ 0 = **no better than random labeling**
- ✓ Negative = **worse than random**

- ✓ Unlike the Silhouette Score, ARI **needs true labels** — so it is only usable for validation, never during real unsupervised training
- ✓ A high ARI confirms the algorithm rediscovered structure that matches reality — a reassuring sanity check when learning clustering for the first time
- ✓ In real deployments without true labels, the **Silhouette Score** (or Elbow Method) is what you fall back on

HANDS-ON · PRACTICAL 2

Clustering Demo

Open the notebook: run K-Means on a customer segmentation dataset, and evaluate it with the Silhouette Score and Adjusted Rand Index.



HANDS-ON PRACTICAL — open the notebook and follow along

PART 2 · IMPROVING MODEL INPUTS

Feature Engineering



What Is Feature Engineering?

The process of creating, transforming, and selecting input features from raw data to improve model performance.



Improves Performance

Better features lead to higher accuracy and lower error.



Captures Hidden Patterns

Helps models learn complex relationships more easily.



Reduces Training Time

Well-engineered features simplify the problem.



Improves Interpretability

Meaningful features make the model easier to trust.



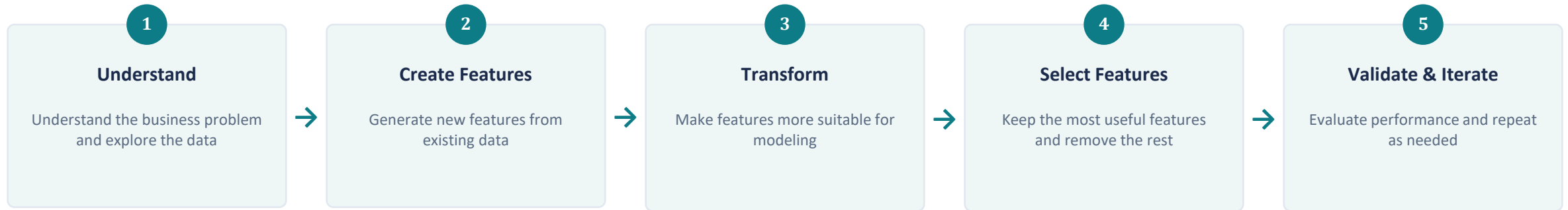
Handles Real-World Complexity

Converts messy, raw data into useful ML input.



Key Takeaway "Garbage in, garbage out" — great features, great models out!

The Feature Engineering Process



Key Takeaway Good features capture the true signal in data, make patterns easier to learn, and improve model performance, generalization, and interpretability.

Common Techniques



Mathematical Transforms

log, exp, sqrt, power, abs



Interactions

Combine features to capture interactions ($A \times B$, A / B , etc.)



Aggregations

Aggregate over groups or time (sum, mean, max, count)



Polynomial Features

Capture non-linear relationships (x^2 , x^3 , $x \cdot y$, ...)



Text Features

Length, word count, TF-IDF, n-grams, sentiment

Examples of Feature Engineering

Type	Description	Example
Creating New Features	Combine or extract new information from existing features.	<code>total_spend = quantity × unit_price</code>
Transforming Features	Change the scale or distribution of features.	<code>log(income)</code> , <code>vage</code> , <code>StandardScaler</code>
Encoding Categorical Features	Convert categories into numerical representation.	One-Hot Encoding, Label Encoding
Binning (Discretization)	Convert continuous variables into bins.	<code>age</code> → [0-18], [19-35], [36-60], [60+]
Date/Time Features	Extract meaningful information from dates.	From '2024-05-20': <code>year</code> , <code>month</code> , <code>day_of_week</code>

Feature Selection & Best Practices

Feature Selection Methods

- ✓ Filter Methods (**model-independent**) — **Correlation, Mutual Information, Chi-Square Test, Variance Threshold**
- ✓ Wrapper Methods (**model-dependent**) — **Recursive Feature Elimination (RFE), Forward/Backward Selection**
- ✓ Embedded Methods — **L1 Regularization (Lasso), Tree-based Feature Importance**

Best Practices

- ✓ Understand the data and domain deeply
- ✓ Create features with a clear purpose
- ✓ Avoid data leakage — use only information available at prediction time
- ✓ Keep features simple and interpretable whenever possible
- ✓ Validate impact using cross-validation

HANDS-ON · PRACTICAL 3

Feature Engineering Demo

Open the notebook: create new features and see their effect on model performance.



HANDS-ON PRACTICAL — open the notebook and follow along

PART 2 • MAKING MODELS RELIABLE

Overfitting, Underfitting & Optimization



Overfitting, Underfitting & Optimization

- **Overfitting:** The model learns the training data too well, including noise and outliers. It performs well on training data but poorly on unseen data.
- **Underfitting:** The model is too simple to capture the underlying pattern. It performs poorly on both training and unseen data.
- **Optimization:** The process of tuning model/learning parameters and hyperparameters to achieve the best possible performance and generalization.



Key Takeaway The goal is to find the right balance — good fit on training data and good generalization on unseen data.

Underfitting vs. Good Fit vs. Overfitting

Underfitting (High Bias)

- ✓ Model is too simple
- ✓ High training error
- ✓ High validation/test error

EXAMPLE

Using a linear model for a non-linear problem

Good Fit (Just Right)

- ✓ Captures the underlying pattern
- ✓ Low training error
- ✓ Low validation/test error

EXAMPLE

Appropriate model complexity

Overfitting (High Variance)

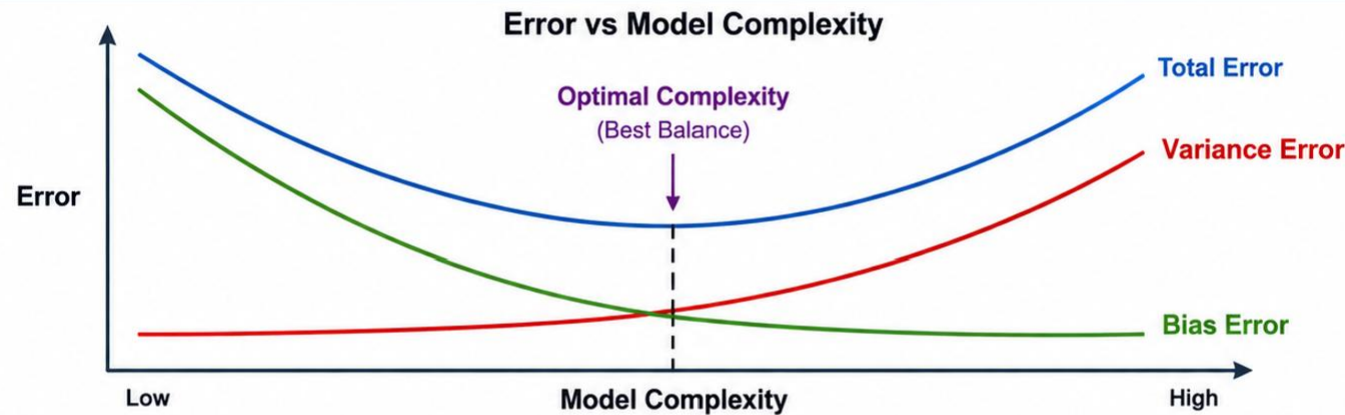
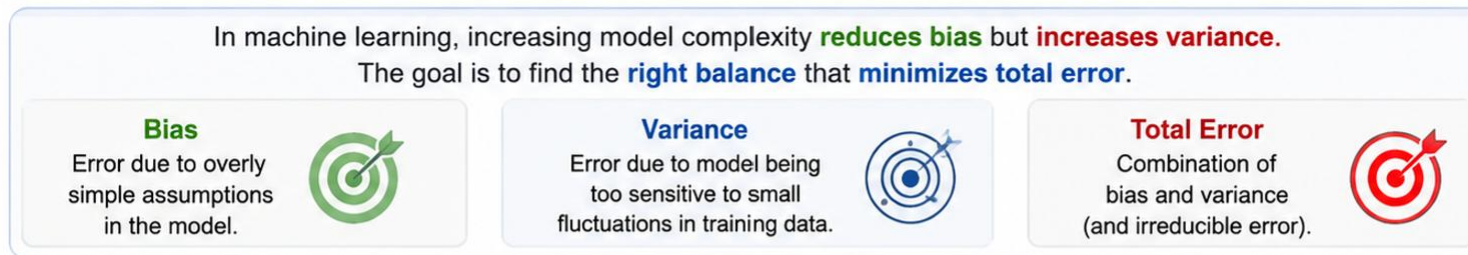
- ✓ Model is too complex
- ✓ Learns noise and fluctuations
- ✓ High validation/test error

EXAMPLE

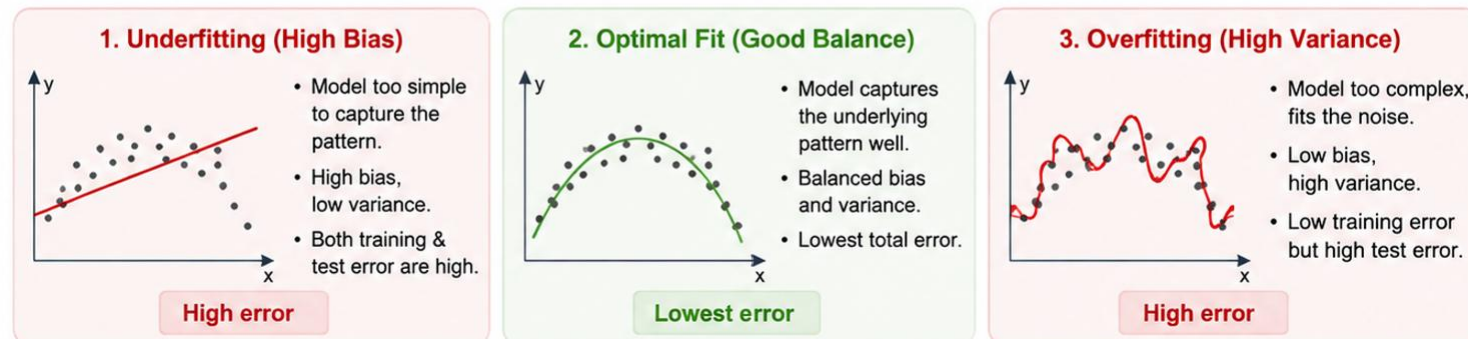
Very deep model / too many parameters

The Bias-Variance Trade-off

As model complexity increases, training error always decreases; validation error first decreases, then increases as the model starts memorizing noise.



$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$



How to Fix It — Practical Tips



More Data

More data helps reduce variance (overfitting).



Simplify Model

Use simpler models, reduce depth or features.



Regularization

Use L1 (Lasso) or L2 (Ridge) to penalize large weights.



Feature Engineering

Use relevant features; remove noise and redundancy.



Cross-Validation

Use k-fold CV to validate model stability.



Early Stopping

Stop training when validation error starts increasing.

Common Optimization Techniques

Grid Search • Random Search • Bayesian Optimization • Gradient Descent — evaluated with Accuracy, Precision/Recall, F1-Score, ROC-AUC, or RMSE/MAE.

What Is Regularization?

A technique used to prevent overfitting by adding a penalty for model complexity, discouraging the model from learning noise in the training data.

$$J_{reg}(\theta) = J(\theta) + \lambda \cdot \Omega(\theta)$$

$J(\theta)$ = original loss (e.g., MSE) • $\Omega(\theta)$ = penalty term • λ = controls regularization strength ($\lambda = 0 \rightarrow$ no regularization)



Reduces Overfitting

Lowers high variance in the model.



Improves Generalization

Performs better on unseen data.



Controls Complexity

Discourages overly complex models.



More Robust

Makes models more stable overall.

L1 (Lasso) vs. L2 (Ridge)

L1 Regularization (Lasso)

$$J(\theta) + \lambda \sum |\theta_i|$$

- ✓ Encourages sparsity (drives some coefficients to exactly 0)
- ✓ Performs feature selection
- ✓ Useful when only a few features matter

L2 Regularization (Ridge)

$$J(\theta) + \lambda \sum \theta_i^2$$

- ✓ Shrinks coefficients towards but never exactly 0
- ✓ Keeps all features but reduces their impact
- ✓ Useful when many features contribute

Effect of Lambda (λ) & Other Methods

Small λ (≈ 0)

Weak regularization — model overfits (high variance)

Moderate λ

Best balance — good fit, best generalization

Large λ

Strong regularization — model underfits (high bias)

Other Regularization Methods



Elastic Net (L1 + L2)

Combines L1 and L2 penalties; useful when features are correlated.



Dropout (Neural Nets)

Randomly drops neurons during training to improve generalization.



Early Stopping

Stops training when validation error stops improving.

Summary & Best Practices

Method	Penalty	Effect on Coefficients	Use When
$L1$ (Lasso)	$\sum \theta_i $	Some become exactly 0 (sparse)	Feature selection needed
$L2$ (Ridge)	$\sum \theta_i^2$	Shrink towards 0 (never exact)	Keep all features
Elastic Net	$ \theta_i + \theta_i^2$	Balance between L1 and L2	Correlated features

Best Practices

- ✓ Standardize features before applying regularization
- ✓ Use cross-validation to choose the right value of λ
- ✓ Monitor training and validation error curves
- ✓ Prefer simpler models that generalize well

Cross-Validation: A More Reliable Check

A single train-test split can be lucky or unlucky. K-Fold Cross-Validation gives a more reliable estimate of real-world performance.



Teal = used for training this round. Amber = held out for validation this round.

- ✓ The data is split into K equal folds (5 shown above)
- ✓ The model trains on K – 1 folds and validates on the remaining fold
- ✓ This repeats K times, so every fold gets a turn as the validation set
- ✓ The K validation scores are averaged into one overall, more trustworthy estimate

Key Takeaways

- ✓ Decision Trees split data using questions that maximize Information Gain (or minimize Gini/Entropy)
- ✓ Random Forest combines many trees trained on random samples and features, reducing overfitting through voting or averaging
- ✓ K-Means clustering groups unlabeled data by minimizing within-cluster spread, evaluated with the Silhouette Score (and Adjusted Rand Index when ground truth exists)
- ✓ Feature Engineering (binning, interactions, polynomial and log transforms) can make patterns easier for models to learn
- ✓ The bias-variance tradeoff explains underfitting and overfitting; regularization (L1/L2) and cross-validation help keep models reliable
- ✓ Gradient descent and loss functions are the universal training mechanism behind models large and small, including modern GenAI systems

Thank You

Questions & Discussion

This completes the Basic Machine Learning series — well done!

